JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent.atomic

Class AtomicReference<V>

java.lang.Object
  java.util.concurrent.atomic.AtomicReference<V>

Type Parameters:

`V - The type of object referred to by this reference`

All Implemented Interfaces:

`Serializable`

```
public class AtomicReference<V>
extends Object
implements Serializable
```

An object reference that may be updated atomically. See the `java.util.concurrent.atomic` package specification for description of the properties of atomic variables.

Since:

`1.5`

See Also:

`Serialized Form`

Constructor Summary

Constructors

Constructor	Description
`AtomicReference()`	Creates a new AtomicReference with null initial value.

`AtomicReference(V initialValue)`	Creates a new AtomicReference with the given initial value.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`V`	`accumulateAndGet(V x, BinaryOperator<V> accumulatorFunction)`	Atomically updates the current value with the results of applying the given function to the current and given values, returning the updated value.
`boolean`	`compareAndSet(V expect, V update)`	Atomically sets the value to the given updated value if the current value == the expected value.
`V`	`get()`	Gets the current value.
`V`	`getAndAccumulate(V x, BinaryOperator<V> accumulatorFunction)`	Atomically updates the current value with the results of applying the given function to the current and given values, returning the

		previous value.
V	getAndSet(V newValue)	Atomically sets to the given value and returns the old value.
V	getAndUpdate(UnaryOperator<V> updateFunction)	Atomically updates the current value with the results of applying the given function, returning the previous value.
void	lazySet(V newValue)	Eventually sets to the given value.
void	set(V newValue)	Sets to the given value.
String	toString()	Returns the String representation of the current value.
V	updateAndGet(UnaryOperator<V> updateFunction)	Atomically updates the current value with the results of applying the given function, returning the updated value.
boolean	weakCompareAndSet(V expect, V update)	Atomically sets the value to the given updated value

		if the current value == the expected value.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

AtomicReference

```
public AtomicReference(V initialValue)
```

Creates a new AtomicReference with the given initial value.

Parameters:

`initialValue - the initial value`

AtomicReference

```
public AtomicReference()
```

Creates a new AtomicReference with null initial value.

Method Detail

get

```
public final V get()
```

Gets the current value.

Returns:

`the current value`

set

```
public final void set(V newValue)
```

Sets to the given value.

Parameters:

`newValue - the new value`

lazySet

```
public final void lazySet(V newValue)
```

Eventually sets to the given value.

Parameters:

`newValue - the new value`

Since:

1.6

compareAndSet

```
public final boolean compareAndSet(V expect,
                                   V update)
```

Atomically sets the value to the given updated value if the current value == the expected value.

Parameters:

`expect - the expected value`

`update - the new value`

Returns:

`true if successful. False return indicates that the actual value was not equal to the expected value.`

weakCompareAndSet

```
public final boolean weakCompareAndSet(V expect,
                                       V update)
```

Atomically sets the value to the given updated value if the current value == the expected value.

May fail spuriously and does not provide ordering guarantees, so is only rarely an appropriate alternative to `compareAndSet`.

Parameters:

`expect - the expected value`

`update - the new value`

Returns:

`true if successful`

getAndSet

```
public final V getAndSet(V newValue)
```

Atomically sets to the given value and returns the old value.

Parameters:

`newValue - the new value`

Returns:

`the previous value`

getAndUpdate

```
public final V getAndUpdate(UnaryOperator<V> updateFunction)
```

Atomically updates the current value with the results of applying the given function, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`updateFunction - a side-effect-free function`

Returns:

`the previous value`

Since:

`1.8`

updateAndGet

```
public final V updateAndGet(UnaryOperator<V> updateFunction)
```

Atomically updates the current value with the results of applying the given function, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`updateFunction - a side-effect-free function`

Returns:

`the updated value`

Since:

1.8

getAndAccumulate

```
public final V getAndAccumulate(V x,
                                BinaryOperator<V> accumulatorFunction)
```

Atomically updates the current value with the results of applying the given function to the current and given values, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. The function is applied with the current value as its first argument, and the given update as the second argument.

Parameters:

`x - the update value`

`accumulatorFunction - a side-effect-free function of two arguments`

Returns:

`the previous value`

Since:

`1.8`

accumulateAndGet

```
public final V accumulateAndGet(V x,
                                BinaryOperator<V> accumulatorFunction)
```

Atomically updates the current value with the results of applying the given function to the current and given values, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. The function is applied with the current value as its first argument, and the given update as the second argument.

Parameters:

`x - the update value`

`accumulatorFunction - a side-effect-free function of two arguments`

Returns:

`the updated value`

Since:

`1.8`

toString

```
public String toString()
```

Returns the String representation of the current value.

Overrides:

`toString` in class `Object`

Returns:

the String representation of the current value

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.